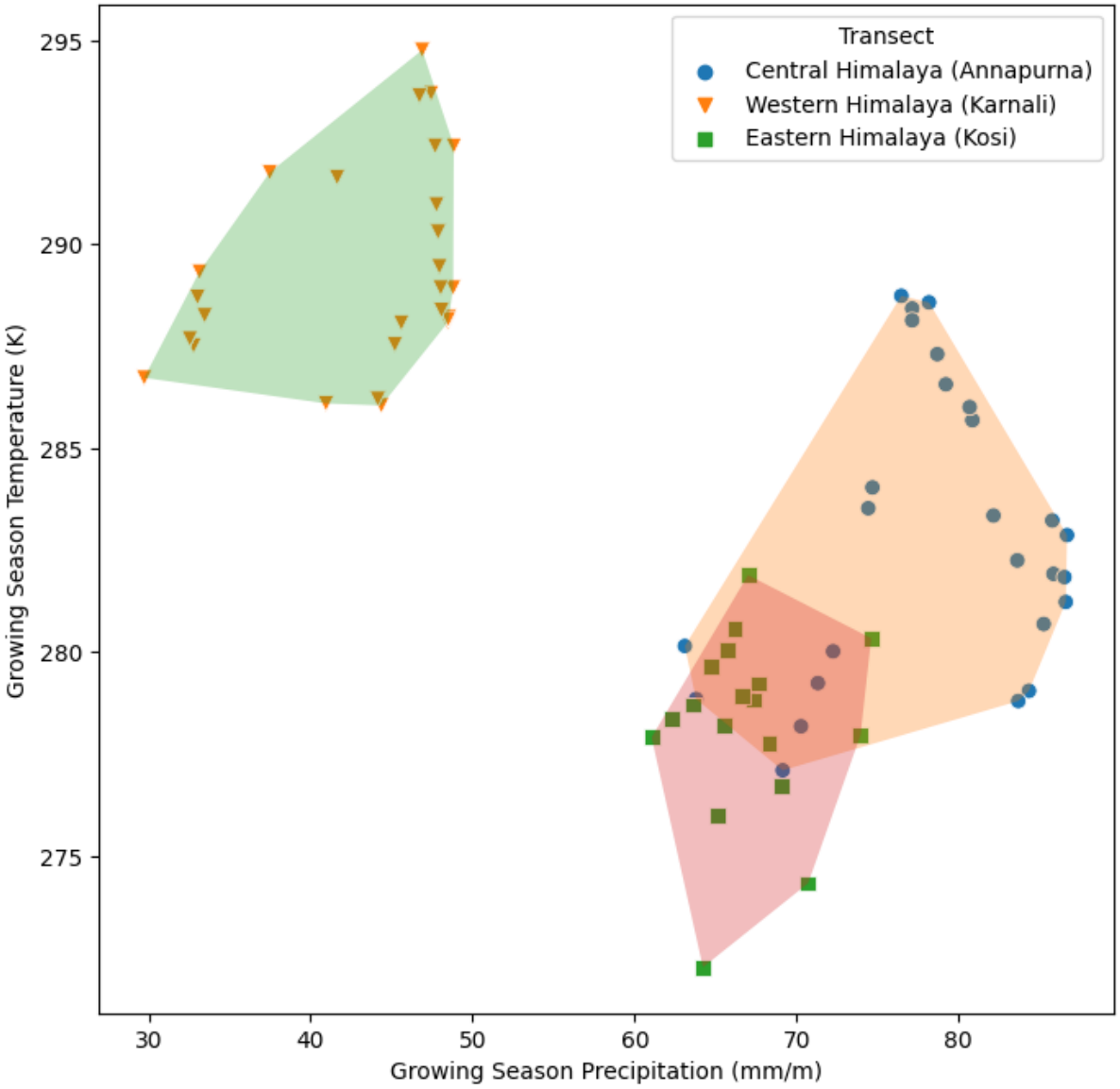
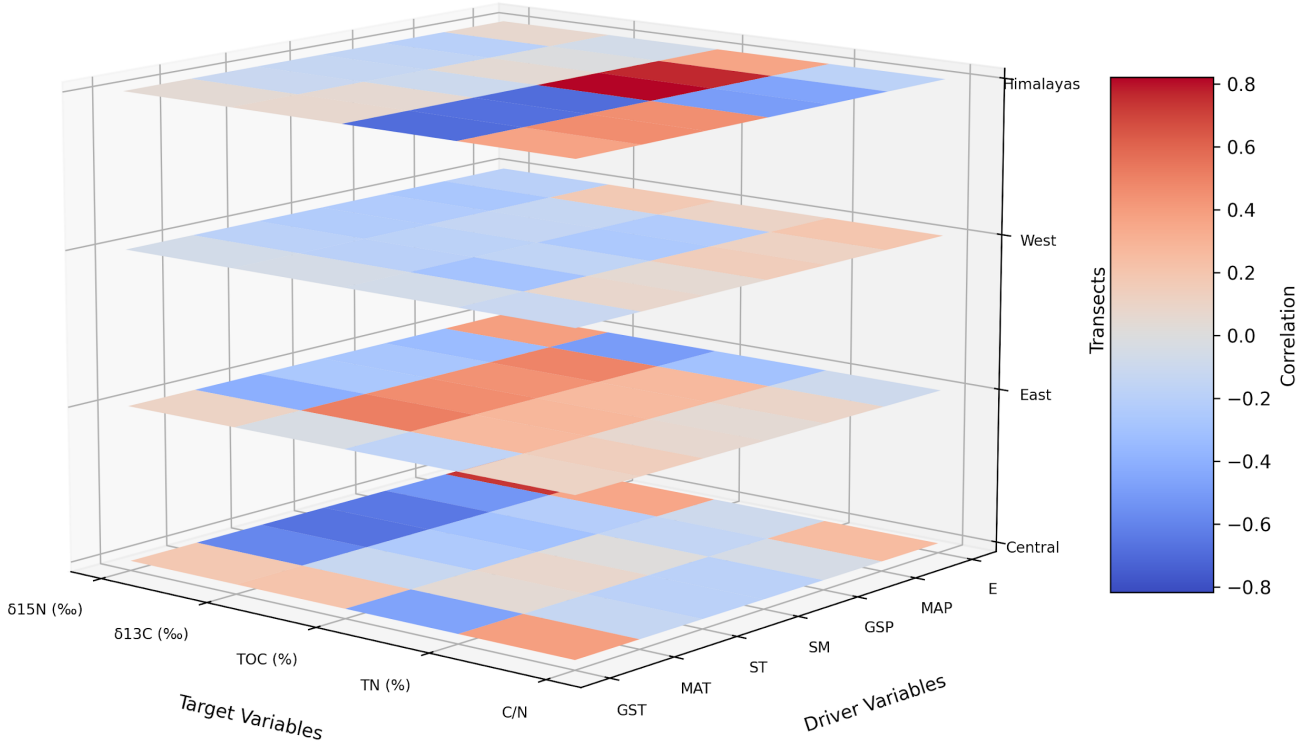
Supplementary I: Figures

Figure S1: Climatological distinction between the 3 transects, with a dry-to-wet and hot-to-cold gradient from the west and east





5 **Figure S2: 3D Heatmap of Pearson Correlation between Drivers and Target Variables categorised by Transects and the whole study area**

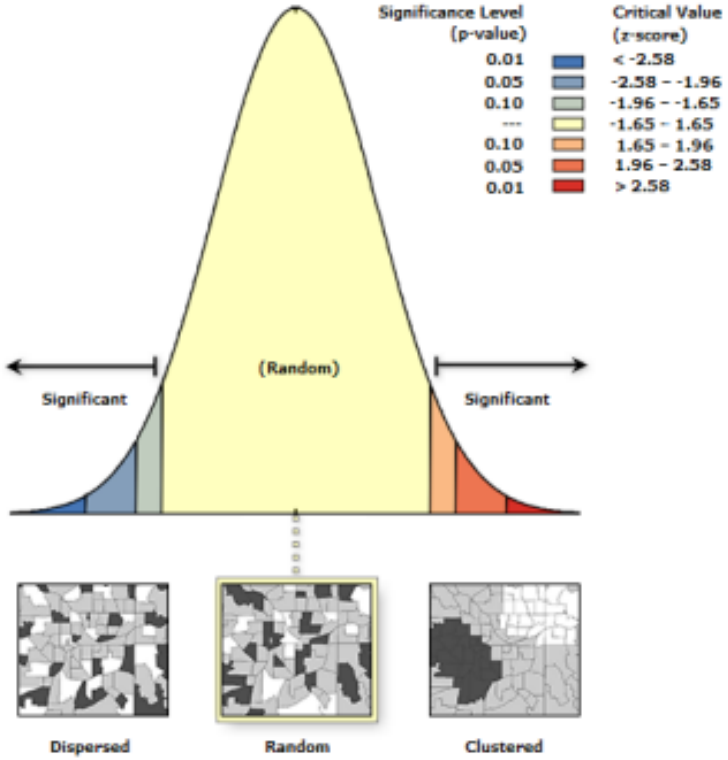


Figure S3 (a): Moran's I for Foliar $\delta^{13}C$. Given the z-score of 1.0406, the pattern does not appear to be significantly different than random.

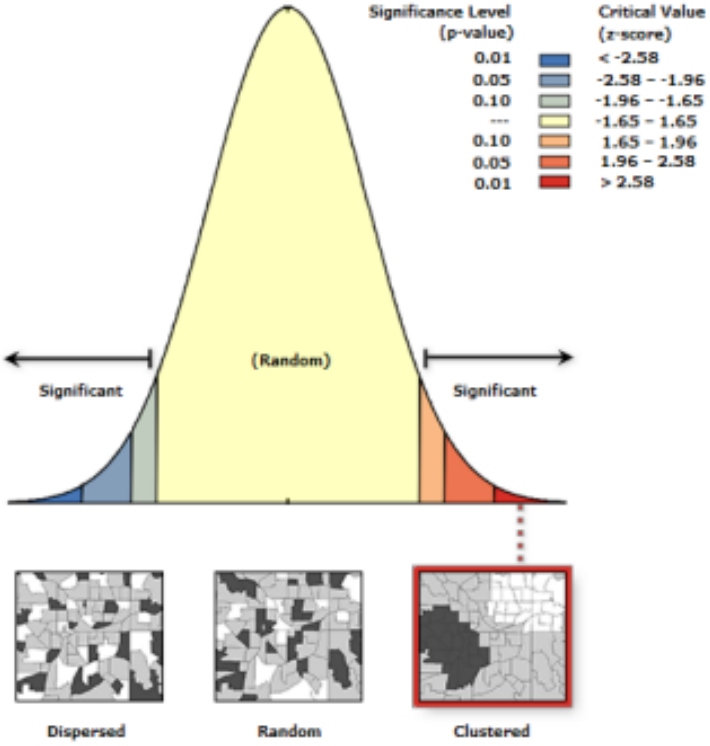


Figure S3 (b): Moran's I for Foliar $\delta^{15}N$. Given the z-score of 9.662064, there is a less than 1% likelihood that this clustered pattern could be the result of random chance.

Supplementary II: Tables (attached separately as .xlsx file with all 3 tables)

Supplementary III: Python code for Genera-Mycorrhizal clustering

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import silhouette_score, adjusted_rand_score
import matplotlib.pyplot as plt
from itertools import combinations

# Function to evaluate and visualize clustering for a given dataframe and features
def evaluate_clustering(df, features, true_label_column, k=4):
    # Extract the relevant columns
    X = df[features]

    # Standardize the features
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Perform clustering
    kmeans = KMeans(n_clusters=k, random_state=0)
    df['Cluster'] = kmeans.fit_predict(X_scaled)

    # Evaluate clustering using Silhouette Score
    silhouette_avg = silhouette_score(X_scaled, df['Cluster'])
    print(f'Silhouette Score for features {features}: {silhouette_avg}')

    # Evaluate clustering against true labels using Adjusted Rand Index
    label_encoder = LabelEncoder()
    true_labels = label_encoder.fit_transform(df[true_label_column])
    adjusted_rand = adjusted_rand_score(true_labels, df['Cluster'])
```

```
    print(f'Adjusted Rand Index for features {features}: {adjusted_rand}')

    # Visualize the clusters and label association
    plt.figure(figsize=(8, 6))
    scatter = plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=df['Cluster'], cmap='viridis')

    plt.xlabel(f'{features[0]} (Standardized)')
    plt.ylabel(f'{features[1]} (Standardized)')
    plt.title(f'Clustering Results with {true_label_column} Labels')
    plt.grid()

    # Create a legend for cluster colors
    handles, _ = scatter.legend_elements()
    plt.legend(handles, df[true_label_column].unique(), title=true_label_column)
    plt.show()

    return silhouette_avg, adjusted_rand

# First condition: Mycorrhizal association filtering
options_1 = ['Endomycorrhizal', 'Ectomycorrhizal', 'Arbuscular mycorrhiza', 'Ericoid']
df_filtered_1 = data.loc[data['Mycorrhizal association'].isin(options_1)]

# Evaluate clustering for δ13C (‰) and TN (%)
evaluate_clustering(df_filtered_1, ['δ13C (‰)', 'TN (%)'], 'Mycorrhizal association')

# Second condition: Domain filtering
options_2 = ['Gymnosperm', 'Angiosperm', 'Grass C3', 'Grass C4']
df_filtered_2 = data.loc[data['Domain'].isin(options_2)]

# Define the features you want to consider in combinations
feature_columns = ["TOC (%)", "δ13C (‰)", "TN (%)", "δ15N (‰)", "C/N"]

# Initialize lists to store the results
results = []

# Generate dual feature combinations and evaluate clustering for each combination
for comb in combinations(feature_columns, 2):
```

```
        silhouette_avg, adjusted_rand = evaluate_clustering(df_filtered_2, list(comb), 'Mycorrhizal
    association')
        # Append the results to the list
        results.append({
            'Features': ', '.join(comb),
            'Silhouette Score': silhouette_avg,
            'Adjusted Rand Index': adjusted_rand
        })

    # Create an output DataFrame and display it
    output_df = pd.DataFrame(results)
    print(output_df)
```